

Agentes de código y SDD greenfield

Lección 1 – Spec-Driven Development

Construir software confiable con agentes, de a una spec por vez.

Qué vamos a ver hoy

Tiempo	Tema
0:00 – 0:30	¿Por qué SDD? – "anda" vs "cumple el requerimiento"
0:30 – 1:00	Agentes de código 101 – loop, contexto, tools, subagents
1:00 – 1:15	Break
1:15 – 1:50	Refinar requerimientos, diseño y arquitectura
1:50 – 2:20	El ciclo SDD (greenfield), de punta a punta
2:20 – 2:50	Escribir una spec que un agente pueda implementar
2:50 – 3:00	Briefing de la tarea (en equipo)

Al final de esta clase podés...

Describir qué es un agente de código – y en qué difiere del autocompletado y del chat

Detectar los modos de falla del prompting suelto

Usar un agente para dar forma a requerimientos, diseño y arquitectura

Enunciar el principio SDD: requerimientos + verificación *antes* del código

Mapear el flujo greenfield de punta a punta

¿Por qué SDD?

"anda en mi máquina" $\neq$ "cumple el requerimiento"

"Hacéme un login"

¿Qué pediste realmente? El agente tiene que *adivinar*:

¿Email y contraseña?

¿OAuth?

¿Magic link?

¿Bloqueo al 5° intento?

¿Reglas de contraseña?

¿Flujo de recupero?

¿Duración de sesión?

¿Qué ve el usuario?

El agente llena cada hueco con una **suposición plausible** – y entrega algo que corre, demuestra bien, y está silenciosamente *mal*.

«Anda» responde a...

¿corrió?

«Cumple el requerimiento» responde a...

¿hizo lo que hacía falta?

Tres modos de falla del prompting suelto



Ambigüedad

El agente inventa detalles que nunca acordaste. Los encontrarás en la revisión – o en producción.



Drift

En un chat largo el contexto se corre. Lo que aprobaste queda pisado sin que te enteres.



Salida no verificable

"Hacelo rápido." No hay nada que chequear, entonces no hay nada en qué confiar.

Los agentes **amplifican** los tres – más velocidad hacia el objetivo equivocado.

La respuesta de SDD

Dejá de tratar la **conversación** como fuente de verdad. Trátá la **spec** como fuente de verdad.



Requerimientos sin ambigüedad



Criterios de verificación, antes del código



Un **plan iterativo**



El agente ejecuta contra el **contrato**

Principio SDD: requerimientos + criterios de verificación, *antes* de implementar.

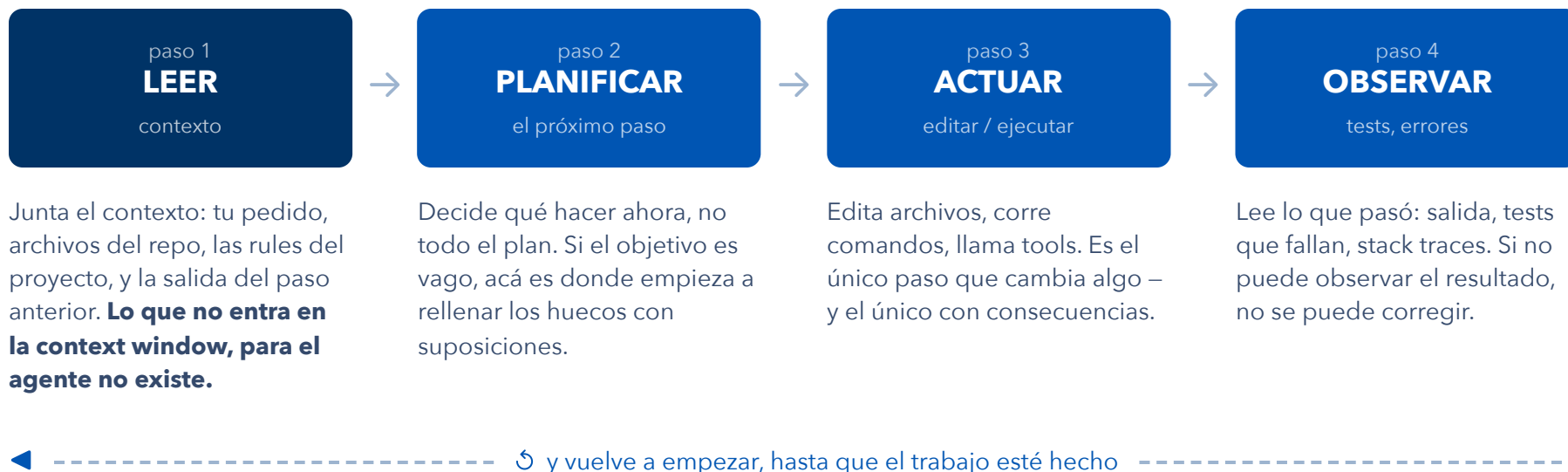
Agentes de código 101

El loop del agente, contexto, tools y subagents

Autocompletado → Chat → Agente

	Autocompletado	Chat	Agente
Alcance	el próximo token/línea	una respuesta	tarea multi-paso
Lee tu repo	el archivo actual	lo que le pegás	el codebase
Actúa	–	sugiere	edita archivos, corre comandos
Se verifica	–	–	observa resultados, reintenta

El loop del agente



La calidad de cada vuelta depende de **Leer**. Contexto basura adentro, acciones basura afuera.

Qué alimenta el loop

Context window

La memoria de trabajo – finita. Lo que no está adentro, no existe.

Tools

Leer/escribir archivos, correr comandos, buscar, llamar sistemas.

Skills

Playbooks en `SKILL.md` – el pipeline SDD entero es uno.

Rules

Restricciones persistentes del proyecto, aplicadas en cada sesión.

Subagents

Delegados para tareas puntuales – mantienen liviano el contexto principal.

Demo: agente vs. chat, misma tarea

Chat

- Le pegás fragmentos → te devuelve prosa
- Copiás el código a los archivos a mano
- No sabés si compila ni si pasa
- El contexto vive en *tu* cabeza

Agente

- Lee el repo por su cuenta
- Escribe directo en los archivos
- Corre los tests, ve que fallan, corrige, reintenta
- Itera hasta terminar de verdad

CHECKPOINT

¿Qué falló?

Le pedís al agente "agregá exportar a CSV". Lee el repo, escribe el código, corre los tests: todo verde. Al día siguiente el PM avisa que faltaban las columnas de fecha y que el separador tenía que ser ; . ¿Qué pasó?

A Drift: en un chat largo el contexto se corrió y pisó lo acordado. ❌

No había nada acordado que pisar: el pedido fue una sola línea, sin ningún detalle.

B Ambigüedad: el agente llenó los huecos con suposiciones plausibles. ✅

Columnas y separador nunca se dijeron. El agente no se frena ante lo que falta: lo rellena al PLANIFICAR.

C Faltaron tests: con tests, el agente lo habría detectado solo. ❌

Hubo tests, y pasaron: verificaban lo que el agente supuso, no lo que hacía falta.

D No es un agente: le faltaba la tool para correr comandos. ❌

Sí corrió los tests: cerró el loop. El problema estuvo antes, en qué se le pidió.

💡 "Anda" ≠ "cumple el requerimiento". Lo que no está escrito, el agente lo adivina.

Refinar requerimientos, diseño y arquitectura

Antes de que corra el pipeline – construir el *base context*

No especifiques desde una hoja en blanco

Usá el agente como **socio para pensar**: convertí una idea vaga en materia prima sólida **antes** de escribir la primera línea de la spec.

1 • QUÉ PROBLEMA

Requerimientos

¿Qué problema resuelve? ¿Para quién?
¿Qué entra y qué queda afuera? ¿Qué restricciones hay?

2 • CÓMO, Y POR QUÉ ASÍ

Diseño

Explorá opciones y trade-offs, no la primera que funciona. **Anotá lo que descartaste y por qué.**

3 • QUÉ FORMA TIENE

Arquitectura

La forma gruesa: componentes, flujo de datos, actores y dónde están los límites entre ellos.

Acá se diverge. Preguntás, explorás, descartás. Todavía no te comprometés con nada.

Esto no es la spec. Es la materia prima con la que se construye.

Idea vaga → base context

Los mismos tres de recién, ahora sobre un caso concreto. Cada paso se apoya en el anterior.



El agente hace las preguntas que te olvidarías. **Vos seguís decidiendo.**

Base context – por qué importa

La misma suposición equivocada, según dónde la encuentres:



cuanto más tarde la encuentras, más cara sale



Le da a **Especificar** una ventaja: propósito, alcance y actores ya listos



Separa **pensar** de **especificar**: divergí acá, comprométe después

El ciclo SDD (greenfield)

spec → revisión → plan → implementar → verificar → freeze

Greenfield: no hay legacy contra el que pelear

Greenfield = construir desde cero. No hay código existente que te condicione, ni regresiones de las que cuidarte.

Por eso mismo aprendemos el pipeline acá primero – nada se interpone entre vos y el flujo de trabajo.

El **base context** que acabás de construir se convierte en el **punto de partida** de la spec. Desde ahí, el trabajo avanza por una secuencia fija de pasos.

El pipeline de seis pasos

La salida de cada paso es la entrada del siguiente.



Dos siglas que vas a ver todo el curso, y que salen del paso 1:

FR – requerimiento funcional: algo que el sistema tiene que *hacer*.

VC – criterio de verificación: el chequeo observable que demuestra que ese FR se cumple.

El único paso que vuelve atrás es **Revisar**. Todo lo demás avanza.

Cómo se encadenan los pasos

- **Especificar** → el base context se vuelve un contrato verificable
- **Revisar** → lo habilita; nada de planificar con preguntas abiertas
- **Planificar** → iteraciones autocontenidas, cada una con criterios de éxito
- **Implementar** → una iteración por vez – *acá se contiene el drift*
- **Verificar** → demuestra que cada criterio de verificación (VC) se ejercita, de punta a punta
- **Freeze** → destila la spec entregada en una **context capsule**

Por qué iterar y no big-bang

Big-bang



un solo chequeo al final – el drift se descubre tarde

Iterativo



verificás cada paso – el desvío se detecta en la iteración 1

El plan es una secuencia de contratos, cada uno más chico y chequeable. El agente no puede irse muy lejos si tiene que probar cada paso.

CHECKPOINT

¿Planificamos igual?

Terminaste la spec de un CLI de tareas. Revisar devuelve **no está lista**: quedan dos preguntas abiertas, ¿se permiten títulos duplicados? y ¿dónde se guarda el archivo? Tu compañero propone: "planifiquemos igual, lo resolvemos cuando implementemos". ¿Qué hacés?

A Planificar y dejar las dos preguntas anotadas como **TBD** en el plan. ❌

Un TBD en el plan es un TBD en el código: termina resolviéndolo el agente, en silencio.

C Planificar solo las iteraciones que no dependen de esas dos preguntas. ❌

Dónde se guarda el archivo toca casi todas las iteraciones. Partir el plan alrededor del hueco solo lo esconde.

B **Volver a Especificar**, resolver las dos preguntas y pasar de nuevo por Revisar. ✅

Revisar es el gate, el único paso que vuelve atrás. No se planifica contra una spec con preguntas abiertas.

D Pasarle la spec al agente y que elija la opción más razonable. ❌

Es el modo de falla "ambigüedad" con otro nombre: decide alguien que después no va a defender la decisión.

💡 Resolver la duda en la spec cuesta **un párrafo**. Después de implementar, **una reescritura**.

Escribir una spec que un agente pueda implementar

Propósito, alcance, actores, FRs, BRs, NFRs, VCs

Anatomía de una spec

Una spec tiene dos partes: el **contexto**, que enmarca, y los **requerimientos**, que son el contrato.

 **Propósito** – por qué existe

 **Alcance** – qué entra y qué queda afuera

 **Actores** – quién interactúa

FR · requerimiento funcional

Qué tiene que **hacer** el sistema, como comportamiento observable.

ej. El sistema rechaza una tarea con título duplicado.

BR · regla de negocio

La **política** que lo restringe. Lleva fundamento y excepciones.

ej. El título tiene entre 1 y 200 caracteres.

NFR · requerimiento no funcional

Cuán bien lo hace. Siempre con un número y una condición.

ej. `list` responde en < 100 ms con 10.000 tareas.

VC · criterio de verificación

El **chequeo observable** que demuestra que se cumple.

ej. Agregar un duplicado sale con código 1 e imprime «duplicate».

Cada FR y cada BR tiene un VC. Si no lo podés verificar, no lo especificaste.

FRs en Dado / Cuando / Entonces

Un FR es **atómico** y **verificable**. Dado/Cuando/Entonces explicita el disparador, la acción y el resultado – *incluyendo los caminos de falla*.

Cada FR tiene su **VC**: el chequeo observable que lo demuestra.

FR-3: Rechazar tareas duplicadas

Dado que ya existe una tarea

"comprar leche",

Cuando el usuario agrega

"comprar leche" de nuevo,

Entonces el sistema la rechaza
con un error y la lista queda
sin cambios.

VC-3: Agregar un duplicado

devuelve exit code 1, imprime

"duplicate", y la cantidad

almacenada no cambia.

BRs y NFRs

Reglas de negocio (BRs) – política + fundamento + excepciones

BR-1: El título de una tarea tiene entre 1 y 200 caracteres. *Fundamento*: almacenamiento y visualización. *Excepción*: ninguna.

Requerimientos no funcionales (NFRs) – un umbral medible

✗ "El CLI debería ser rápido."

✓ NFR-1: `list` responde en < 100 ms con hasta 10.000 tareas.

Anti-patrones a evitar

Resultados vagos – "manejar los errores con elegancia". No verificable $\Rightarrow$ no especificado.

Caminos de falla ausentes – solo el happy path. ¿Input inválido? ¿Estado vacío? ¿Límites?

"TBD" sin resolver – resolvelo, o lo decide el agente por vos.

FRs sin VC – un requerimiento que no podés chequear es un deseo.

Si quien revisa no puede decidir si lo construido satisface una línea, esa línea no está terminada.

CHECKPOINT

¿Qué línea está lista?

Tu compañero te pasa la spec de un **acortador de URLs** para revisar. ¿Cuál de estas líneas pasa la revisión tal como está?

A FR-2: El sistema maneja los errores de red con elegancia. ❌

Resultado vago: ¿qué es "con elegancia"? Nadie puede decidir si lo construido lo cumple.

B NFR-1: La redirección tiene que ser rápida. ❌

NFR sin número: faltan métrica, umbral y carga. "Rápido" es una opinión, no un test.

C FR-4: Dado un código inexistente, Cuando se visita /abc,  Entonces responde 404. **VC-4:** GET /abc devuelve 404 y no crea registros.

Atómico, cubre un camino de falla y trae un VC observable: cualquiera puede chequearlo.

D BR-1: Largo máximo del código corto: TBD. ❌

TBD sin resolver: si no lo decidís vos, lo decide el agente por vos.

 Si quien revisa no puede decidir si una línea se cumple, **esa línea no está terminada.**

Planificar e implementar con contexto

De la spec al código en iteraciones que se pasan el estado por archivos

Planificar: de la spec al plan

El plan se escribe al lado de la spec: `gcsgrep.md` → `gcsgrep-plan.md`. Solo se planifica una spec que **pasó Revisar** sin preguntas abiertas.

Cada iteración se escribe para un lector que **no estuvo antes**: un agente con la ventana en blanco que solo conoce la spec, el plan y lo que quedó en disco.

Reglas que el plan impone:

- **Gate** – la iteración N+1 no arranca hasta verificar los criterios de N
- **Incremento funcional** – cada iteración deja algo corriendo y testeable, nunca solo scaffolding
- **Un commit por iteración**, código + artefactos juntos
- **Sin estado huérfano** – si se interrumpe, el repo sigue compilando
- **Docs al final** – la última iteración escribe la documentación para humanos

```
## Iteración 2: Búsqueda literal bajo un prefijo

**Goal**: `gcsgrep PATRON gs://bucket/prefijo`
lista objeto:línea según spec §4.1.

**Reads**: Artifact Protocol 1-5 +
src/gcsgrep/client.py

**Scope**:
- Listado de objetos, streaming, match literal
- Artifacts: CONTEXT.md (file map,
  interfaces), DECISIONS.md si hubo
  elección, iterations/02-*.md

**Success criteria**:
- VC-003: `pytest tests/search -q` → 8 passed
- VC-004: objeto binario → se saltea con aviso

**Hands off**: firma de `iter_objects()`,
invariante "nunca se escribe en el bucket"
```

El contrato de artefactos

Las iteraciones se hablan **solo por archivos**. El plan define el directorio y qué va en cada uno.

```
gcsgrep-plan-artifacts/  
├── CONTEXT.md           # estado vivo – se reescribe  
├── DECISIONS.md         # log de decisiones – solo append  
└── iterations/  
    ├── 01-cli-skeleton.md # handoff immutable  
    ├── 02-busqueda.md  
    └── 03-flags.md
```

Cada iteración **lee primero** spec → plan → **CONTEXT.md** → **DECISIONS.md** → últimos 2 records, y **escribe último**, antes del commit.

CONTEXT.md – "¿dónde está el proyecto hoy?"

Estado actual, file map, interfaces públicas, convenciones, comandos copy-pasteables, huecos conocidos. Se *sobrescribe*, así nunca queda viejo. < 300 líneas.

DECISIONS.md – "¿qué ya se decidió?"

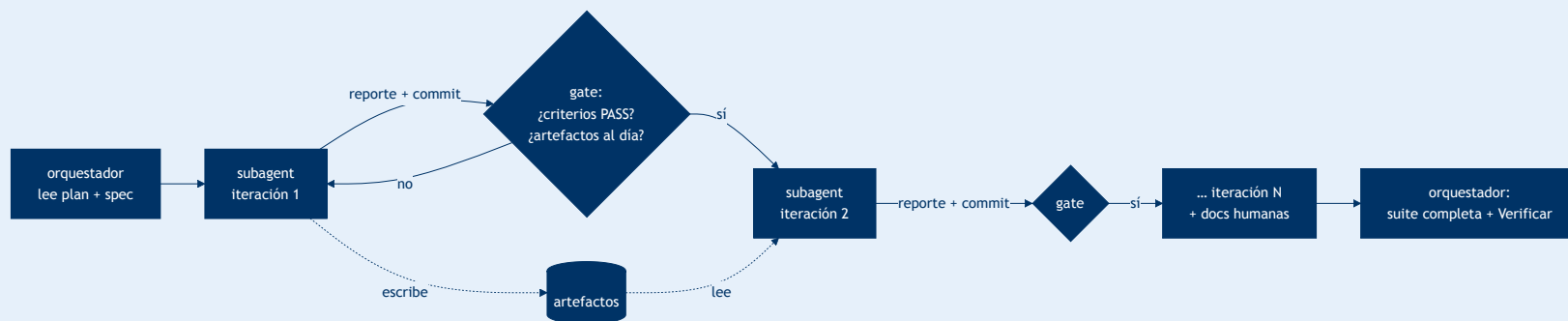
Una entrada por elección no obvia: contexto, decisión, alternativas rechazadas, consecuencias. Nadie vuelve a discutir lo cerrado.

iterations/NN-*.md – "¿qué pasó en la N?"

Criterios PASS/FAIL con el comando, SHA, desvíos, gotchas y "para la próxima iteración". Se escribe una vez, no se edita.

Implementar: un agente por iteración

El agente principal es **orquestador**: lee el plan, despacha un subagent por iteración, verifica y avanza. No escribe código de iteración.



El brief del subagent

Rutas absolutas a spec, plan y artefactos; su iteración citada textual; las reglas como restricciones; formato de reporte exigido.

El gate

Todo PASS y commiteado → chequear que `iterations/NN` existe y `CONTEXT.md` está en el commit. Artefactos viejos = FAIL, aunque el código esté verde.

Bloqueo

Scope ambiguo o criterio que no pasa → el subagent se detiene *sin commitear* y reporta. El orquestador te pregunta, nunca elige en silencio.

¿Por qué mantener el contexto entre iteraciones?

Cada subagent arranca con la ventana **vacía**. Eso es una ventaja, no un problema – si el estado vive en disco.

😞 Una sesión larga

Iteración 6 arrastra los callejones sin salida de la 1 a la 5. La ventana se llena de ruido, la spec se desdibuja, el drift crece. Si se corta, se pierde todo.

🕒 Sesiones nuevas sin memoria

Cada iteración redescubre el proyecto, reabre decisiones cerradas, rompe convenciones que no conoce. Contexto limpio pero *amnésico*.

📁 Contexto fresco + estado en disco

Cada iteración lee `CONTEXT.md` y sigue desde ahí. Ventana chica y enfocada, memoria completa y curada. Lo mejor de los dos.

🧹 **Ventana limpia por iteración** – el barro de explorar se tira con la ventana; solo cruza lo destilado

🔄 **Reanudable** – mañana, otro modelo u otra persona retoma en la iteración 4

📁 **Convenciones estables** – el file map y los invariantes viajan, no se reinventan

🔒 **Decisiones que no se relitigan** – `DECISIONS.md` cierra lo cerrado

🔍 **Auditable** – cada iteración deja qué corrió, qué pasó y qué cambió respecto al plan

📖 **Docs gratis al final** – la última iteración sintetiza los artefactos en documentación humana

Conceptos clave – repaso

-  **Base context** – requerimientos/diseño/arquitectura, refinados primero
-  **Spec como contrato** – verificás contra la spec, no contra el chat
-  **Criterios de verificación** – observables, medibles, uno por FR/BR
-  **Planes iterativos** – iteraciones autocontenidas con criterios propios
-  **Greenfield** – sin legacy; ideal para aprender el pipeline completo

Briefing de la tarea

Lab greenfield – para hacer en casa, en equipo de trabajo

Tu tarea: gcsgrep

Llevá la **experiencia de grep a Google Cloud Storage** – buscar dentro del *contenido* de objetos remotos sin bajarlos primero.

La idea

Un CLI que hace grep sobre objetos de texto de un bucket/prefijo – `gcsgrep "timeout" gs://logs/` – y muestra qué objeto matcheó, y dónde.

Lo que te damos

`examples/01-greenfield/tarea/` – un borrador **deliberadamente subespecificado**: FRs flojos, BRs candidatos, NFRs en blanco, preguntas abiertas. Con su consigna.

Dice **qué** queremos, no **cómo**. Convertir ese borrador vago en un contrato verificable es el ejercicio.

Este es tu **base context** – refinalo, no lo implementes tal cual.

Cómo encarar gcsgrep

1 • Refiná el base context

Usá el agente para resolver las **preguntas abiertas**:

¿sabor de regex?

¿auth (ADC / key)?

¿direccionamiento `gs://?`

¿qué flags de grep?

¿binarios / `.gz`?

¿guardrails de costo?

2 • Corré el pipeline en serio

Especificar → Revisar → Planificar → Implementar → Verificar – con umbrales reales en los NFRs y una tabla de cobertura de VCs.

v1 sugerida (discutila): búsqueda literal o regex básica sobre objetos de texto bajo un prefijo de un bucket, con `-i`, `-n`, salida con nombre de objeto, lectura por streaming, solo lectura, y exit codes estilo grep. Todo lo demás → iteraciones posteriores.

Mantenelo **read-only**, nunca amplíes el acceso más allá de las credenciales de quien lo invoca, y no escanees un bucket enorme sin un guardrail.

Tarea (para antes de la Lección 2)

Trabajen en su **equipo de trabajo**. Lleven `gcsgrep` de borrador subespecificado a una Iteración 1 spec'ada y verificada.



Spec revisada

≥ 3 FRs con sus VCs, umbrales reales en los NFRs, preguntas abiertas resueltas



`gcsgrep-plan.md`

≥ 2 iteraciones; el alcance diferido vive acá



Código de la Iteración 1

corre una búsqueda real, los chequeos de verificación pasan

Corran el pipeline en serio: **Especificar → Revisar → Planificar → Implementar → Verificar**

Entrega y presentación



Mail a la cátedra con el **link al repo** del equipo y el **hash del commit**



Hasta el **jueves al mediodía** de la semana anterior a la Lección 2



Al inicio de la Lección 2 se **sortea un equipo** para presentar



Miramos: calidad de la spec, cobertura de VCs, base context → build

Vengan todos listos para mostrar: el sorteo es en el momento. La consigna completa está en `examples/01-greenfield/tarea/enunciado.md`.

¿Preguntas?

Próxima: Lección 2 – SDD brownfield

Cambiar un codebase que ya existe sin perder el control.